

Lezione 21 - JDBC(Java Database Connectivity)

Da Java Base a Java Avanzato

Nei moduli precedenti abbiamo costruito le fondamenta del linguaggio: sintassi, tipi, strutture di controllo, OOP (classi, interfacce, ereditarietà), e i meccanismi di concorrenza con il multithreading. Abbiamo anche incontrato alcuni design pattern fondamentali, tra cui lo Strategy (separare l'algoritmo dal contesto che lo usa) e l'Adapter (fare da ponte tra interfacce incompatibili).

Con Java Avanzato entriamo nell'ecosistema reale delle applicazioni enterprise:

- comunicazione con database,
- gestione delle risorse, integrazione con sistemi esterni.

Non si tratta di nuovi costrutti sintattici, ma di **API e architetture** che poggiano esattamente sui concetti già visti — e JDBC ne è l'esempio perfetto.

JDBC — Java Database Connectivity

DBC è un'API standard di Java per la comunicazione con database relazionali.

Il suo ruolo non è implementare la connessione, ma **definire un contratto**:

- **un insieme di interfacce e regole che qualsiasi driver per qualsiasi DBMS (PostgreSQL, MariaDB, Oracle, MySQL...) deve rispettare.**

🔗 È un'applicazione diretta del pattern Adapter

JDBC astrae le differenze tra i vari database esponendo un'unica interfaccia uniforme verso il codice Java, mentre ogni driver si occupa di tradurre quella interfaccia nel protocollo specifico del suo DBMS .

La comunicazione è però **parzialmente** standardizzata.

La struttura delle chiamate Java è sempre la stessa, ma il **dialetto SQL** sottostante può variare:

- Oracle, ad esempio, ha funzioni e sintassi che differiscono dallo standard SQL usato da PostgreSQL o MariaDB. JDBC non risolve questo problema — garantisce uniformità nell'**accesso**, non nella **query**.

Il punto di ingresso è il `DriverManager` :

- il componente che, dato un URL di connessione (es. `jdbc:postgresql://localhost:5432/mydb`), individua il driver corretto tra quelli registrati e restituisce una `Connection`.

Obiettivo di JDBC

L'obiettivo principale di JDBC è:

- permettere di sviluppare applicazioni Java che si interfaccino con qualsiasi DBMS senza dipendere da una scelta specifica.

Il codice Java che usa JDBC è scritto una volta sola:

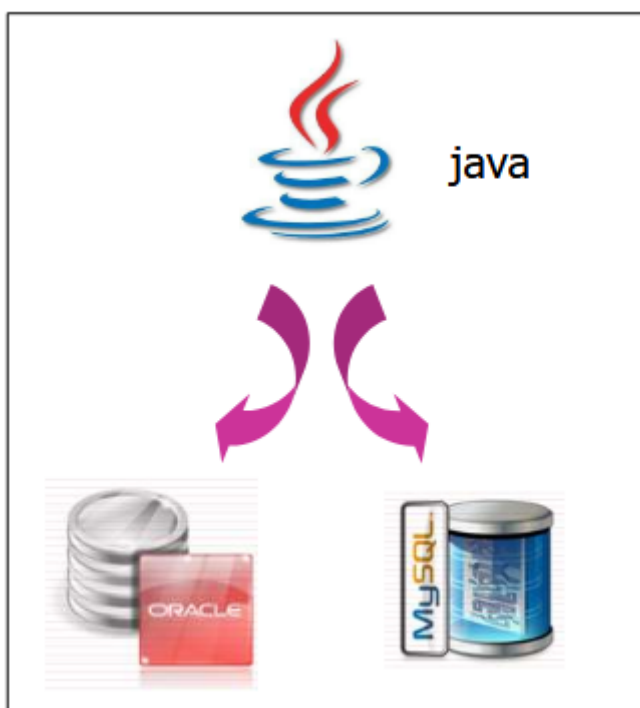
- se un domani si decide di passare da MySQL a PostgreSQL, l'unica cosa che cambia è il driver (e l'URL di connessione) — il codice applicativo rimane invariato.

🔗 In termini pratici:

impatto zero sul codice Java in caso di migrazione tra DBMS.

Questo è possibile proprio perché JDBC non è un'implementazione, ma un contratto:

- le classi e interfacce del package `java.sql` definiscono cosa si può fare, mentre ogni driver definisce come farlo per il suo specifico database.



Cos'è JDBC — nel dettaglio

JDBC è:

- un'interfaccia a basso livello verso il database.
- Non offre astrazioni di alto livello (quelle arriveranno con ORM come Hibernate), ma accesso diretto ed esplicito tramite SQL.

Si trova nel package standard `java.sql` e copre quattro funzionalità fondamentali:

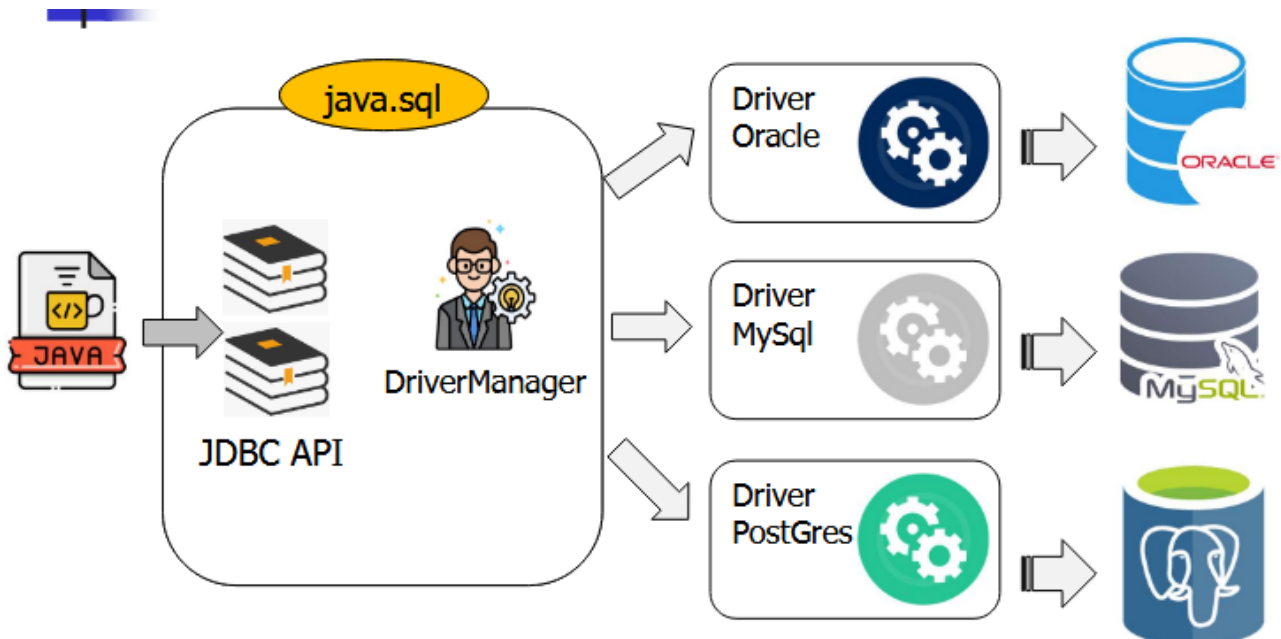
1. **Connettersi al database** — aprire una connessione verso il DBMS
2. **Eseguire scritture** — `INSERT` , `UPDATE` , `DELETE`
3. **Eseguire query** — `SELECT`
4. **Recuperare i risultati** — leggere i dati restituiti dalla query riga per riga

Nota:

"basso livello" qui significa che sei tu a scrivere le query SQL per esteso, a gestire la connessione, a iterare sui risultati. Non c'è magia — ed è esattamente questo che lo rende utile da studiare prima di passare a strumenti più astratti.

Architettura JDBC

Per capire come Java e JDBC lavorino insieme, è utile seguire il flusso di una connessione dall'inizio alla fine.



Analisi dell'immagine:

Il codice Java applicativo (il file Java sulla sinistra) non comunica mai direttamente con il database:

- **parla esclusivamente con le interfacce definite nel package `java.sql` (`Connection`, `Statement`, `ResultSet`, ecc.).**
- **Questo è il confine che il codice applicativo non attraversa mai.**

Mentre all'interno di `java.sql` vive il `DriverManager`:

- **come abbiamo già anticipato sopra questo è il componente che fa da punto di ingresso.**
Quando gli si fornisce un URL di connessione come `jdbc:postgresql://localhost:5432/mydb`, il `DriverManager` **legge il prefisso del protocollo e individua automaticamente il driver corretto tra quelli disponibili, restituendo una `Connection`.**

I **driver** invece vivono **fuori** da `java.sql`:

- sono librerie fornite da terze parti (il team di PostgreSQL, Oracle, ecc.) e ognuno di essi è un'**implementazione concreta** delle interfacce di `java.sql`.

🔗 **È esattamente il pattern Adapter:**

i driver adattano il protocollo specifico del loro DBMS al contratto comune definito da JDBC.

Il risultato è che:

- **il codice applicativo è completamente ignaro di quale DBMS si trovi dall'altra parte.**

Per migrare da PostgreSQL a MariaDB è sufficiente cambiare il driver e l'URL di connessione — il codice Java rimane invariato.

I Driver JDBC

I driver:

- **sono classi Java che permettono di lavorare con un database specifico.**

Come abbiamo visto nell'architettura, JDBC definisce le interfacce e i fornitori

di database (PostgreSQL, Oracle, ecc.) ne forniscono l'implementazione concreta.

Esistono 4 tipi di driver, in ordine crescente di "purezza" Java:

Tipo 1 — JDBC-ODBC Bridge

Il più datato, oggi obsoleto.

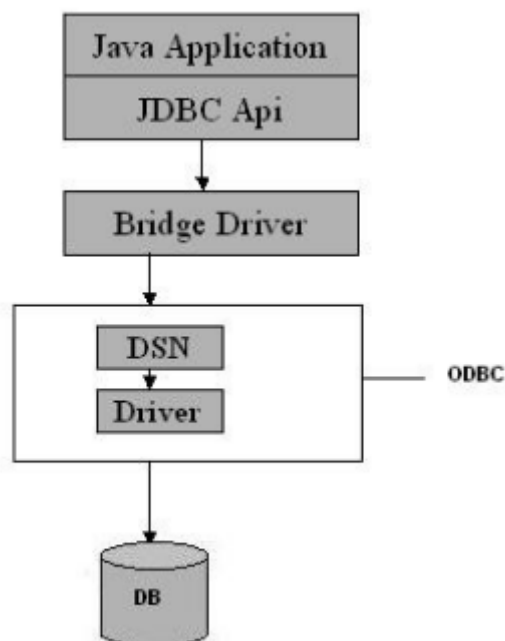
Fa da ponte verso ODBC:

- una famiglia di driver nativa di Windows che permette di connettersi a database come Microsoft Access.

È implementato dalla classe `sun.jdbc.odbc.JdbcOdbcDriver`, fornita con il Java 2 SDK.

Il limite principale è la **dipendenza dalla piattaforma**:

- funziona solo su Windows, e si usava solo quando non era disponibile un driver Java nativo per un certo database.



Tipo 2 — API Nativa

Utilizza le API native disponibili direttamente sulla macchina client (quella dove risiede il database).

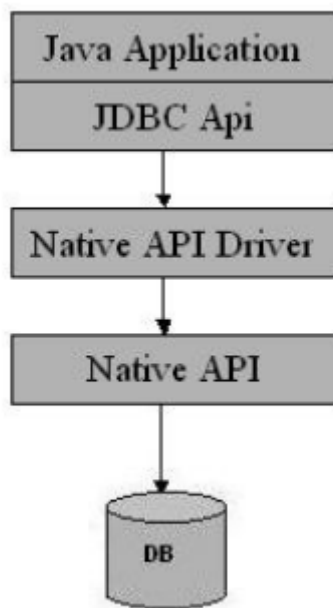
Per questo motivo non è scritto interamente in Java e dipende dal sistema operativo — ad esempio Oracle fornisce driver nativi di questo tipo.

✓ **Vantaggi**

Il vantaggio rispetto al Tipo 1 è che elimina il sovraccarico delle chiamate ODBC, offrendo **più funzionalità e performance migliori**.

⚠ **Svantaggi**

Tuttavia, per applicazioni che richiedono interoperabilità tra sistemi diversi, è preferibile il Tipo 4.



Tipo 3 — Protocollo di Rete

Introduce uno strato intermedio (middle-tier) che si occupa di tradurre le chiamate JDBC verso il database.

Il middle-tier può a sua volta usare un driver di Tipo 1, 2 o 4 per comunicare col DBMS effettivo.

È scritto **interamente in Java**:

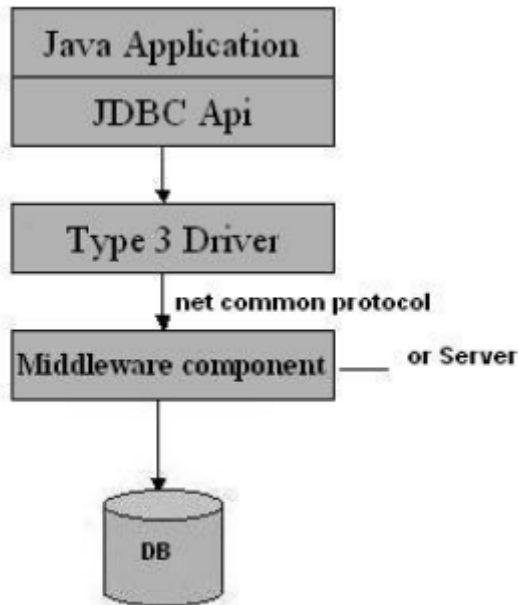
- **quindi pienamente interoperabile, e può essere configurato per supportare diversi database.**

✓ **Vantaggi**

Offre ottime performance, maggiore sicurezza e funzionalità avanzate come caching e load balancing.

× Svantaggi

Il contro è che **richiede un server dedicato** per il middle-tier.



Tipo 4 — Protocollo Nativo ✓ (quello che useremo)

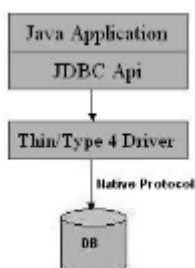
È il driver più moderno e quello che utilizzeremo. **Comunica direttamente con il database usando le API di Java networking, senza alcuno strato intermedio.**

È scritto interamente in Java, quindi completamente interoperabile, ed è dedicato a un database specifico (es. il driver JDBC di PostgreSQL funziona solo con PostgreSQL).

È il più **leggero** tra tutti i driver proprio perché non richiede nulla di aggiuntivo:

- nessun software lato client,
- nessun server intermedio come nel Tipo 3,
- nessun bridge come nel Tipo 1.

Basta aggiungere la dipendenza al progetto e il driver è pronto.



Caricare i Driver JDBC

I driver JDBC vengono caricati **dinamicamente** dall'applicazione leggendo il classpath.

È però possibile caricarli esplicitamente con:



Java

```
1 Class.forName("com.mysql.cj.jdbc.Driver"); // driver nativo MySQL
8
2 Class.forName("sun.jdbc.odbc.JdbcOdbcDriver"); // driver bridge
  (obsoleto)
```

`Class.forName()` :

- carica la classe del driver in memoria, rendendolo disponibile al `DriverManager`.
- Una volta caricato, la connessione avviene tramite i metodi statici di `DriverManager`.

Nota:

Da Java 4 in poi, la chiamata a `Class.forName()` è **opzionale** — **il driver viene rilevato automaticamente dal classpath tramite il meccanismo di *Service Provider Interface (SPI)*.**

I Tipi Chiave di JDBC

Tipo	Natura	Ruolo
<code>DriverManager</code>	classe	Gestore dei driver; fornisce la <code>Connection</code> dato un URL
<code>Connection</code>	interfaccia	Rappresenta la connessione attiva al DB
<code>Statement</code>	interfaccia	Wrapper per l'esecuzione di statement SQL
<code>ResultSet</code>	interfaccia	Accesso ai risultati di una query

Nota:

che `Connection`, `Statement` e `ResultSet` sono tutte **interfacce** — non classi. È il driver (Tipo 4, nel nostro caso) a fornire l'implementazione concreta di ognuna di esse.

DriverManager — Dal Concetto al Codice

Come abbiamo visto, il `DriverManager` gestisce i driver registrati e crea le connessioni ai database. Dal punto di vista applicativo, il metodo principale che useremo è `getConnection()`:



Java

```
1 Connection connection = DriverManager.getConnection(url,
    username, password);
```

oppure, se il database non richiede autenticazione:

```
1 Connection connection = DriverManager.getConnection(url, "", "");
```

`getConnection()` è un **metodo statico** — non si istanzia il `DriverManager`, lo si usa direttamente.

La Connection String

L'`url` (detta anche **connection string**) segue sempre questo formato:



Java

```
1 jdbc:nomeDriver:location
```

Ad esempio:



Java

```
1 jdbc:postgresql://localhost:5432/mydb
2 jdbc:mysql://localhost:3306/mydb
```



Java

```
1 Connection connection = DriverManager.getConnection(url,
    username, password);
```

È proprio leggendo il prefisso `nomeDriver` che il `DriverManager` sa quale driver utilizzare tra quelli disponibili nel classpath.

Nota:

`getConnection()` dichiara `throws SQLException` — tutte le operazioni JDBC possono lanciare eccezioni che devono essere gestite, cosa che vedremo nel dettaglio a breve.

Creare le connessioni

La firma completa dei metodi disponibili è:



Java

```
1 // senza credenziali
2 static Connection getConnection(String url) throws SQLException;
3
4 // con credenziali
5 static Connection getConnection(String url, String userId, String
    password) throws SQLException;
```

Esempi pratici



Java

```
1 // Tipo 1 - ODBC (obsoleto): non ha location, solo un'etichetta
  configurata su Windows
2 Connection conn =
  DriverManager.getConnection("jdbc:odbc:SAMPLEDBASE");
3
4 // Tipo 2/4 - Oracle con credenziali
5 Connection conn =
  DriverManager.getConnection("jdbc:oracle:oci7:@sample", "scott",
    "tiger");
6
7 // Tipo 4 - PostgreSQL con credenziali
8 Connection conn =
  DriverManager.getConnection("jdbc:postgresql://localhost:5432/mydb
    "admin", "secret");
9
10 // Se non serve autenticarsi, si passano stringhe vuote
```

```
11 Connection conn =  
    DriverManager.getConnection("jdbc:postgresql://localhost:5432/mydb  
    "", "");
```

Struttura della Connection String

L'URL di connessione, detta anche **connection string**, segue sempre questo formato:

```
1 jdbc : nomeDriver : location
```

dove:

- `jdbc` — **prefisso fisso, identifica il protocollo**
- `nomeDriver` — **identifica il driver da usare (es. `postgresql`, `mysql`, `odbc`)**
- `location` — **indica dove si trova il database (host, porta, nome del DB)**

Eccezione — ODBC (Tipo 1):

non ha una `location` di rete vera e propria, ma una semplice etichetta configurata a livello di sistema operativo durante il setup del driver (es. `SAMPLEDBASE`).

Se il database non richiede autenticazione, si passano stringhe vuote al posto delle credenziali:



Java

```
1 DriverManager.getConnection("jdbc:postgresql://localhost:5432/mydb  
    "", "");
```

 **Nel caso di ODBC (Tipo 1) non esiste una vera `location` di rete — al suo posto si usa una **etichetta** configurata a livello di sistema operativo durante la fase di setup del driver.**

Interfaccia Connection

Una `Connection` rappresenta una **sessione di dialogo attiva** con un database specifico. Non è solo un "canale aperto" — è il **contesto** entro cui avvengono tutte le operazioni SQL.

Le sue responsabilità principali sono due:

1. Creare gli Statement SQL

- È attraverso la `Connection` che si creano gli oggetti per eseguire query e scritture sul DB:



Java

```
1  java.sql.Connection interface
2  void close() throws SQLException;
3  Statement createStatement() throws SQLException;
4  PreparedStatement prepareStatement(String) throws SQLException;
5  CallableStatement prepareCall(String) throws SQLException;
```

2. Gestire le transazioni:

- La `Connection` controlla il ciclo di vita delle transazioni:



Java

```
1  boolean getAutoCommit() throws SQLException;
2
3  void setAutoCommit(boolean) throws SQLException; // disabilita il
    commit automatico
4  void commit() throws SQLException; // conferma la transazione
5  void rollback() throws SQLException; // annulla la transazione
```

Info:

Di default, ogni operazione SQL viene committata automaticamente (`autoCommit = true`). Disabilitarlo permette di raggruppare più operazioni in un'unica transazione atomica — o tutto va a buon fine, o si annulla tutto.

Interfaccia `Statement`

Uno `Statement` è:

- **il componente che permette di eseguire istruzioni SQL sul database.** Si ottiene sempre a partire da una `Connection` e può fare due cose:
- **modificare i dati**
- **o interrogarli.**



Java

```
1  java.sql.Statement interface
2
3  // Per scritture: INSERT, UPDATE, DELETE – restituisce il numero
   di righe modificate
4  int executeUpdate(String sql) throws SQLException;
5
6  // Per letture: SELECT – restituisce un ResultSet con i risultati
7  ResultSet executeQuery(String sql) throws SQLException;
8
9  // Generico: utile quando non si sa a priori il tipo di
   istruzione
10 boolean execute(String sql) throws SQLException;
11
12 // Chiude lo statement e libera le risorse
13 void close() throws SQLException;
```



Nota:

La distinzione tra `executeUpdate()` e `executeQuery()` riflette una separazione netta tra **scrittura** e **lettura** — non si può usare `executeQuery()` per una `INSERT`, né `executeUpdate()` per una `SELECT`.

`executeUpdate()`

Si usa per eseguire istruzioni SQL di **scrittura**:

- `INSERT`, `UPDATE`, `DELETE`.
- **Restituisce un `int` che rappresenta il numero di righe coinvolte nell'operazione.**



Java

```
1  // tramite l'oggetto connection creo lo Statement e lo referenzio
   con un oggetto di Statement
2  Statement st = connection.createStatement();
3
4  // INSERT – restituisce 1 se la riga è stata inserita
5  int res1 = st.executeUpdate("INSERT INTO impiegati VALUES
   ('francesco', 1000, 'programmer')");
6
```

```
7 // UPDATE - restituisce il numero di righe modificate
8 int res2 = st.executeUpdate("UPDATE impiegati SET salario =
  1500");
```

Il valore restituito è utile per verificare che l'operazione abbia effettivamente modificato dei dati:

- **se ritorna 0, nessuna riga è stata coinvolta.**

`executeQuery()`

Si usa per eseguire istruzioni SQL di **lettura**: `SELECT` .

- **Restituisce un oggetto `ResultSet`, ovvero un cursore per navigare riga per riga la tabella risultato.**



Java

```
1 Statement st = connection.createStatement();
2 ResultSet res = st.executeQuery("SELECT * FROM impiegati");
```

⚠ **Ordine delle Colonne nel `ResultSet`**

L'ordine delle colonne restituite da una `SELECT *` **non è garantito** — dipende dal DBMS e dalla struttura interna della tabella, e può variare.

Per garantire un ordine preciso, occorre **specificare esplicitamente** le colonne nella query:



Java

```
1 // ❌ ordine non garantito
2 ResultSet res = st.executeQuery("SELECT * FROM impiegati");
3
4 // ✅ ordine garantito
5 ResultSet res = st.executeQuery("SELECT >nome, cognome,
  salario FROM impiegati");
```

✓ **Questo è anche una buona pratica in generale:**

specificare le colonne rende il codice più leggibile, meno fragile a modifiche future della struttura della tabella, e più efficiente evitando di trasferire dati non necessari.

Interfaccia ResultSet

Un `ResultSet` rappresenta:

- il **risultato tabulare** di una query `SELECT` .
Si comporta come un **Iterator**:
- **parte da una posizione iniziale fuori dalla tabella (riga 0), e si sposta una riga alla volta tramite** `next()` .



Java

```

1  java.sql.ResultSet interface
2
3  boolean next() throws SQLException;           // avanza alla riga
    successiva, false se non ce ne sono
4  void close() throws SQLException;             // chiude il
    ResultSet e libera le risorse
5
6  // Accesso per indice (più efficiente – usare quando si
    specificano le colonne nella SELECT)
7  String getString(int columnIndex) throws SQLException;
8
9  int getInt(int columnIndex) throws SQLException;
10 Date getDate(int columnIndex) throws SQLException;
11
12
13 // Accesso per nome colonna
14 String getString(String columnName) throws SQLException;
15 int getInt(String columnName) throws SQLException;
16 Date getDate(String columnName) throws SQLException;
17
18 // Accesso generico – utile quando il tipo della colonna non è
    noto a compile time
19 Object getObject(int columnIndex) throws SQLException;
20 Object getObject(String columnName) throws SQLException;
21

```

Nota:

L'indice delle colonne parte da 1, non da 0 come gli array Java

Perché farsi restituire un oggetto anziché un intero o un numero?

Come possiamo vedere esistono due metodi che restituiscono un oggetto anziché una Stringa o un intero.

Perché questo e a cosa ci può servire?

Immaginiamo di dover farci restituire il valore di una singola colonna ma non sappiamo a compile-time che tipo ha quella colonna, questo perché magari stiamo scrivendo un tool generico che legge qualsiasi tabella.

Far eseguire il `getString()` o il `getInt()` senza sapere il tipo di dato incastonato nella riga può essere rischioso perché ci obbliga a fare un cast

specifico, che non sempre è il cast corretto, per questo si sceglie il `getObject()` perché **è più generico e se non si sa che tipo di dato si sta estraendo dalla singola riga è la soluzione migliore.**

Un altro motivo è quando si vuol gestire un qualsiasi tipo di colonna con lo stesso codice senza scrivere un `if` per ogni tipo possibile:



Java

```
1 // invece di fare:
2 if (tipo == "String") res.getString("colonna");
3 else if (tipo == "int") res.getInt("colonna");
4
5 // puoi fare genericamente:
6 Object valore = res.getObject("colonna");
```

Esempio Completo:



Java

```
1 Statement st = conn.createStatement();
2 ResultSet res = st.executeQuery("SELECT nome, cognome, salario
  FROM impiegati");
3
4 while (res.next()) {
5     System.out.println("Nome: " + res.getString("nome"));
6     System.out.println("Cognome: " + res.getString("cognome"));
7     System.out.println("Salario: " + res.getInt("salario"));
8 }
```

Navigare il ResultSet



Java

```
1 Statement st = connection.createStatement();
2 ResultSet res = st.executeQuery("SELECT * FROM impiegati");
3
4 while (res.next()) {
5     System.out.println("Nome: " + res.getString("nome"));
6     System.out.println("Cognome: " + res.getString("cognome"));
7 }
8
```

```
9 st.close();
```

Il `ResultSet` si comporta come un **Iterator**:

- all'inizio il cursore si trova in una posizione fuori dalla tabella (riga 0), prima di qualsiasi riga.
- Ad ogni chiamata di `next()` il cursore avanza di una riga e restituisce `true` ; quando le righe sono esaurite restituisce `false` e il ciclo termina.
- Fissata la riga corrente, si accede ai valori delle singole colonne tramite i metodi `getTipo()` , passando il nome della colonna o il suo indice (che parte da 1):



Java

```
1 res.getString("nome"); // per nome colonna
2 res.getString(1);      // per indice - più efficiente
```

Note Importanti sul `ResultSet`

1. Il `ResultSet` consuma gli elementi:

- Righe e colonne si possono scorrere una sola volta — non è possibile tornare indietro o rileggere una riga già letta (a meno di non usare un `ResultSet` scorrevole, che vedremo dopo).

2. Riuso dello `Statement` :

- Se si esegue una nuova query sullo stesso oggetto `Statement` , il `ResultSet` precedente viene chiuso automaticamente.
- È buona pratica chiudere esplicitamente `ResultSet` e `Statement` quando non servono più.

3. `ResultSetMetaData` :

- Se non si conosce a priori la struttura della tabella (numero di colonne, nomi, tipi), si può recuperare questa informazione tramite `ResultSetMetaData` :



Java

```
1 ResultSetMetaData meta = res.getMetaData();
2 int numColonne = meta.getColumnCount();
3 String nomeColonna = meta洗.getColumnName(1);
```

Interfaccia PreparedStatement

PreparedStatement estende Statement e può essere usato per qualsiasi istruzione SQL.

Il suo vantaggio principale è:

- evitare le noiose concatenazioni di stringhe che si sarebbero necessarie con uno Statement normale.

Immagina di dover inserire un impiegato con dei parametri variabili usando uno Statement classico:



Java

```
1 // ✗ Con Statement - concatenazione fragile e rischiosa
2 String sql = "INSERT INTO impiegati VALUES ('" + nome + "', " +
  salario + ", '" + ruolo + "')";
3 st.executeUpdate(sql);
```

Questo approccio è scomodo e pericoloso perché bisogna gestire manualmente i delimitatori per stringhe (') e date, ed è vulnerabile a SQL Injection.

Con PreparedStatement si usano dei **segnaposto** ? al posto dei valori variabili, e per ognuno si invoca il corrispondente metodo `setTipo(index, value)`:

- index — **posizione del ? nell'istruzione, a partire da 1**
- value — **valore da sostituire al segnaposto**



Java

```
1 // ✓ Con PreparedStatement
2 PreparedStatement ps = connection.prepareStatement(
3     "INSERT INTO impiegati VALUES (?, ?, ?)"
4 );
```

Per ogni ? **si invoca il corrispondente metodo setter, specificando la posizione (che parte da 1) e il valore**:



Java

```
1 ps.setString(1, "Francesco");
2 ps.setInt(2, 1000);
3 ps.setString(3, "programmer");
4 ps.executeUpdate();
```

I setter gestiscono automaticamente i delimitatori per stringhe e date — non serve aggiungerli manualmente.

Al termine si esegue l'istruzione con:

- `executeUpdate()` — per `INSERT`, `UPDATE`, `DELETE`
- `executeQuery()` — per `SELECT`

Esempio — Ricerca per matricola



Java

```
1 public Impiegato cerca(String matricola) {
2     PreparedStatement pst = connection.prepareStatement(
3         "SELECT * FROM impiegati WHERE matricola = ?"
4     );
5
6     pst.setString(1, matricola); // nessun delimitatore
    necessario!
7
8     ResultSet result = pst.executeQuery();
9
10    // la matricola è chiave primaria, quindi al massimo una riga
11    if (result.next()) {
12        // leggo le colonne e creo l'oggetto Impiegato
13        return imp;
14    } else {
15        return null; // oppure sollevo un'eccezione
16    }
17 }
```

Settare i parametri

Per ogni tipo di dato esiste il corrispondente setter:



Java

```
1 java.sql.PreparedStatement interface
```

```
2
3 void setByte(int index, byte value) throws SQLException;
4 void setInt(int index, int value) throws SQLException;
5 void setString(int index, String value) throws SQLException;
6
7 // Per le date si usa il tipo java.sql.Date, non java.util.Date!
8 void setDate(int index, Date value) throws SQLException;
```

⚠ Attenzione!

Attenzione alla distinzione tra `java.sql.Date` e `java.util.Date` — **JDBC** usa il suo tipo `Date` specifico per interfacciarsi correttamente con le date dei DBMS.

Il progetto Maven

Finora abbiamo visto come JDBC funzioni concettualmente, ma per usarlo in un progetto reale serve aggiungere il driver come **dipendenza esterna** — ed è qui che entra in gioco **Maven**.

Maven è uno strumento di **gestione delle dipendenze**:

- **permette di dichiarare le librerie di cui il progetto ha bisogno e si occupa automaticamente di scaricarle dal suo repository centrale, insieme a tutte le loro sotto-dipendenze.**

Le dipendenze si dichiarano nel file `pom.xml` (Project Object Model), il file di configurazione di ogni progetto Maven:

```
</> XML
1 <dependency>
2   <groupId>org.postgresql</groupId>
3   <artifactId>postgresql</artifactId>
4   <version>42.6.0</version>
5 </dependency>
```

✎ Nota:

Aggiungendo questa dipendenza, Maven scarica automaticamente il driver JDBC di PostgreSQL ([Tipo 4](#)) — non serve cercare e aggiungere manualmente il `.jar`.

In sintesi, il flusso è:

1. **Dichiari la dipendenza nel `pom.xml`**
2. **Maven scarica il `.jar` richiesto e tutte le sue sotto-dipendenze**
3. **Il driver è disponibile nel classpath e il `DriverManager` può trovarlo automaticamente**

Creare un Progetto Maven su Eclipse

Struttura del Progetto

Una volta creato il progetto Maven (`New Maven Project` → `Create a simple project`), Eclipse genera automaticamente questa struttura:



Plain text

```
1  myproject/
2  |— src/
3  |   |— main/java/      # sorgenti Java
4  |   |— test/java/      # classi di test
5  |— JRE System Library  # libreria standard Java
6  |— pom.xml             # file di configurazione Maven
```

- `Group Id` — **identifica l'organizzazione o il gruppo di progetti (es. `com.miacompany`)**
- `Artifact Id` — **identifica il singolo progetto (es. `gestione-impiegati`)**

Aggiungere il Driver JDBC nel `pom.xml`

Per usare JDBC con MySQL, si aggiunge la dipendenza al connettore nel `pom.xml` :



XML

```
1  <dependencies>
2    <dependency>
3      <groupId>com.mysql</groupId>
```

```
4         <artifactId>mysql-connector-j</artifactId>
5         <version>8.4.0</version>
6         <scope>compile</scope>
7     </dependency>
8 </dependencies>
```

Maven scaricherà automaticamente i .jar necessari, che compariranno sotto Maven Dependencies nel progetto.

A quel punto sono disponibili sia `java.sql` che il driver di connessione, e il progetto è pronto per connettersi al database.

Esempio pratico: Gestione di una libreria

Ora che abbiamo visto la parte teorica del JDBC passiamo alla parte pratica; ipotizziamo di dover mettere in piedi un sistema di gestione dei libri di una libreria o biblioteca a partire da un database scritto in PostgreSQL.

Ora il codice in JDBC viene organizzato in 3 strati con responsabilità separate, seguendo il DAO pattern:

1. Il componente Database :

- Centralizza la logica di connessione al database. Tutte le credenziali e l'URL sono in un unico posto — se cambiano, si modifica solo qui.

2. LibroDTO — La Copia dell'Entità

- Il **DTO** (Data Transfer Object) è:
 - **la rappresentazione Java dell'entità nel database.**
 - **Non contiene logica — solo i campi della tabella con i relativi getter e setter.**
 - Ha **2** costruttori:
 - uno completo (con `id` , usato quando si legge dal DB)
 - e uno senza `id` (usato quando si vuole inserire un nuovo libro, perché l' `id` viene generato dal DB).

3. LibroDAO — L'Accesso ai Dati

- Il **DAO** (Data Access Object) è:
 - **il componente che mette in comunicazione il database con il resto dell'applicazione.**
 - **Contiene tutti i metodi che eseguono query SQL, restituendo o ricevendo oggetti DTO .**

Perché il DAO pattern?

Prima di guardare il codice, vale la pena capire **perché** questa organizzazione esiste.

Immagina di scrivere tutto il codice SQL direttamente nel `main` :

- **funziona, ma ogni volta che la struttura del database cambia devi cercare le query sparse ovunque nel codice.**
- **Se domani vuoi passare da PostgreSQL a MySQL, devi toccare tutto. Questo è esattamente il problema che il DAO pattern risolve.**

L'idea è semplice:

- **separare la logica di accesso ai dati dal resto dell'applicazione.**
Il resto del codice non deve sapere nulla di SQL, di connessioni, di driver — parla solo con oggetti Java.
È un'applicazione diretta del principio di **Single Responsibility**:
- **ogni componente ha una sola ragione per cambiare.**

Questa separazione si realizza attraverso due figure chiave:

1. Il DTO (Data Transfer Object):

- è la rappresentazione Java di un'entità del database — in questo caso un libro.
- Non contiene logica, solo i dati:
 - **è il "contenitore" che viaggia tra i vari strati dell'applicazione al posto delle righe grezze del database.**

2. Il DAO (Data Access Object):

- **è l'unico componente che "parla SQL".**
- **Riceve e restituisce oggetti DTO, nascondendo completamente al resto dell'applicazione il fatto che ci sia un database dietro.**

1. Componente Database

Il componente Database è il terzo strato, ed è il più semplice ma non meno importante:

- Il suo unico compito è centralizzare la logica di connessione:
 - URL, credenziali e tutto ciò che riguarda il "come connettersi" al database vive qui e solo qui.

🔗 Questo significa che se domani le credenziali cambiano, o si migra da PostgreSQL a MySQL, si tocca un solo file — senza dover cercare stringhe di connessione sparse nel codice.

È anche il motivo per cui le costanti `URL`, `USER` e `PSW` sono dichiarate `private static final`:

- `'private'` — la costante non è accessibile dall'esterno della classe
- `'static'` — appartiene alla classe stessa, non a una sua istanza
- `'final'` — il valore viene assegnato una volta sola e non può essere modificato a runtime
- In sintesi, non devono essere accessibili dall'esterno né modificabili a runtime.



Java

```
1 public class Database {
2     // Si definisce una Stringa che rappresenta L'URL per la
    // connessione al database(in questo caso un DB in PostgreSQL)
3     private static final String URL =
    "jdbc:postgresql://localhost:5432/biblioteca";
4     // Questa stringa rappresenta il nome Utente del DB
5     private static final String USER = "postgres";
6     // Questa stringa rappresenta la password per accedere al DB
7     private static final String PSW = "postgres";
8
9     // siccome getConnection() lancia una checked exception,
10    // siamo obbligati a dichiararla con throws o a gestirla con
    try-catch.
11    // Il compilatore ce lo impone - non possiamo ignorarla.
12    public static Connection getConnection() throws SQLException
13    {
14        // restituisco il risultato della connessione chiamandola
        // tramite il DriverManager e passando le costanti come argomenti
        // della chiamata alla funzione
15        return DriverManager.getConnection(URL, USER, PSW);
16    }
17 }
```

🔗 `getConnection()` è static perché non ha senso istanziare un oggetto `Database` — è un punto di accesso globale alla

connessione, esattamente come DriverManager stesso.

⚠ SQLException è una checked exception:

- il compilatore ti obbliga a gestirla esplicitamente.

È il contrario delle unchecked exception (come NullPointerException o IllegalArgumentException) che non richiedono né throws né try-catch.

2. Componente LibroDTO

Come abbiamo già detto il DTO è lo strato che rappresenta l'entità nel database.

Il componente LibroDTO, quindi rappresenta l'entità Libro nel Database.



Java

```
1 package jdbcFullStack.dto;
2
3 public class LibroDTO {
4
5     private int id;
6
7     private String titolo;
8
9     private String autore;
10
11     private Double prezzo;
12
13     public LibroDTO(int id, String titolo, String autore, Double
prezzo) {
14
15         this.id = id;
16         this.titolo = titolo;
17         this.autore = autore;
18
19         this.prezzo = prezzo;
20     }
21
22     public LibroDTO(String titolo, String autore, Double prezzo)
{
23 }
```

```

24         this.titolo = titolo;
25         this.autore = autore;
26         this.prezzo = prezzo;
27     }
28     public int getId() {
29         return id;
30     }
31
32     public void setId(int id) {
33         this.id = id;
34     }
35     public String getTitolo() {
36         return titolo;
37     }
38
39     public void setTitolo(String titolo) {
40         this.titolo = titolo;
41     }
42
43     public String getAutore() {
44         return autore;
45     }
46
47     public void setAutore(String autore) {
48         this.autore = autore;
49     }
50
51     public Double getPrezzo() {
52         return prezzo;
53     }
54
55     public void setPrezzo(Double prezzo) {
56         this.prezzo = prezzo;
57     }
58 }

```

Analisi:

La classe `LibroDTO`, come possiamo notare, possiede 2 costruttori:

1. `public LibroDTO(int id, String titolo, String autore, Double prezzo) :`

- Questo costruttore inizializza gli attributi della classe, compreso l'attributo `id`.

2. `public LibroDTO(String titolo, String autore, Double prezzo)`

- **Il secondo costruttore invece, inizializza gli attributi escluso l' `id`.**

In questo modo non solo si dà la possibilità di istanziare l'oggetto sia con l'`id` che senza, ma questa logica applicativa a uno scopo ben preciso:

- possiamo già supporre che nel `DAO` ci saranno due metodi principali

1. Un metodo di lettura

2. Un metodo di inserimento

quindi il costruttore senza `id`, si userà quando si vuole inserire un nuovo libro nel Database.

 **l' `id` non esiste ancora perché lo genererà il database tramite `RETURN_GENERATED_KEYS`**

Questo metodo, che analizzeremo a breve nel componente `DAO`, corrisponde al tipo di dato `serial` in SQL/PostgreSQL.

In sostanza quando viene inserito il libro l'attributo `id` viene valorizzato in automatico e viene auto-incrementato per ogni nuovo libro che si inserisce.

Mentre il costruttore **con `id`** si usa quando si **legge** dal database:

- **l' `id` esiste già nella tabella e viene passato direttamente al DTO tramite `rs.getInt("id")`**

3. Componente `LibroDAO`

Il `LibroDAO` è:

- l'unico componente che "parla SQL";
 - **tutto il resto dell'applicazione interagisce solo con oggetti `LibroDTO`, ignaro del fatto che ci sia un database dietro.**

Analizziamo i due metodi principali:

`findAll()` — **Lettura di tutti i libri**



Java

```
1 public static List<LibroDTO> findAll() throws SQLException {
2     String sql = "SELECT id, titolo, autore, prezzo FROM libro
    ORDER BY id";
```

```

3      List<LibroDTO> libri = new ArrayList<>();
4      Connection conn = null;
5      try {
6          conn = Database.getConnection();
7          PreparedStatement ps = conn.prepareStatement(sql);
8          ResultSet rs = ps.executeQuery();
9          while (rs.next()) {
10             LibroDTO l = new LibroDTO(
11                 rs.getInt("id"),
12                 rs.getString("titolo"),
13                 rs.getString("autore"),
14                 rs.getDouble("prezzo")
15             );
16             libri.add(l);
17         }
18     } catch (Exception e) {
19         System.out.println("Errore: " + e.getMessage());
20     } finally {
21         if (conn != null) conn.close();
22     }
23     return libri;
24 }

```

Analisi passo per passo:

1. Si dichiara `conn = null` fuori dal `try` :
 - questo è necessario perché il blocco `finally` deve poter accedere alla reference `conn` per chiuderla.
 - Se fosse dichiarata dentro il `try`, il `finally` non la vedrebbe.
2. Si ottiene la connessione tramite `Database.getConnection()` e si prepara lo `Statement` con la query SQL.

Nota:

le colonne sono specificate esplicitamente — non si usa `SELECT *` perché non garantisce l'ordine delle colonne, come abbiamo visto in precedenza.

2. Si scorre il `ResultSet` riga per riga con `rs.next()` :
 - Per ogni riga si crea un nuovo `LibroDTO` usando il costruttore con `id` — perché stiamo leggendo libri che esistono già nel database, quindi l'`id` è già noto.

3. Il blocco `finally` garantisce che la connessione venga chiusa **in ogni caso** — che il `try` sia andato a buon fine o che sia stata lanciata un'eccezione.
- **La guardia** `if (conn != null)` **evita un** `NullPointerException` **nel caso in cui la connessione non fosse mai stata aperta.**

`insert()` — Inserimento di un nuovo libro



Java

```
1  public boolean insert(LibroDTO l) {
2      String sql = "INSERT INTO libro(titolo, autore, prezzo)
VALUES (?, ?, ?)";
3      try (Connection conn = Database.getConnection();
4          PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
5
6          ps.setString(1, l.getTitolo());
7          ps.setString(2, l.getAutore());
8          ps.setDouble(3, l.getPrezzo());
9
10         int righe = ps.executeUpdate();
11         if (righe == 0) throw new SQLException("Nessuna riga
inserita");
12
13         try (ResultSet keys = ps.getGeneratedKeys()) {
14             if (keys.next()) {
15                 l.setId(keys.getInt(1)); // aggiorna il DTO con
l'id generato dal DB
16                 return true;
17             }
18         }
19         return false;
20     } catch (SQLException e) {
21         System.out.println("Errore: " + e.getMessage());
22         return false;
23     }
24 }
25 }
```

Analisi passo per passo:

1. La `Connection` e il `PreparedStatement` vengono dichiarati direttamente nell'intestazione del `try-with-resources` :

- **Java li chiuderà automaticamente al termine del blocco, senza bisogno di un `finally` esplicito.**
 - **È l'approccio più sicuro perché non si può dimenticare di chiudere le risorse.**
2. La query usa i segnaposto `?` per i tre valori variabili.
- Si noti che `id` non compare nella query;
 - non lo forniamo noi, lo genera il database automaticamente come un `SERIAL` in PostgreSQL.
 - Difatti Il secondo argomento `Statement.RETURN_GENERATED_KEYS` istruisce il driver a rendere disponibile l' `id` generato dopo l'insert.
3. `executeUpdate()` **restituisce il numero di righe inserite:**
- **Se è `0`, significa che l'insert non ha avuto effetto — si lancia una `SQLException` per segnalarlo esplicitamente.**
4. Tramite `getGeneratedKeys()` si recupera l' `id` generato dal database e lo si imposta sul `LibroDTO` con `l.setId()`
- **In questo modo l'oggetto Java viene aggiornato con lo stato reale del database — dopo l'insert, il `LibroDTO` ha un `id` valido e può essere usato nel resto dell'applicazione.**

4. Il `main` — Mettere tutto insieme

Il `main` è il punto di ingresso dell'applicazione e rappresenta bene il vantaggio del DAO pattern:

- **non c'è una sola riga di SQL, non c'è nessuna connessione al database — parla solo con oggetti Java.**



Java

```
1  LibroDAO libroDao = new LibroDAO();
2  LibroDTO libro1 = new LibroDTO("Informatica per tutti", "Rob
del", 9.9);
3  LibroDTO libro2 = new LibroDTO("Fluent FratemScript", "Ciro
Gates", 6.66);
4
5  boolean ok = libroDao.insert(libro1);
6  boolean ok2 = libroDao.insert(libro2);
7
8  if (ok && ok2) {
9      System.out.println("Libro inserito con id: " + libro1.getId()
+ "\nLibro inserito con id: " + libro2.getId());
```

```

10 } else {
11     System.out.println("Libro non inserito");
12 }
13
14 StampaLibri();

```

Analisi passo per passo:

1. Si istanzia un oggetto `LibroDAO` :
 - **necessario perché** `insert()` **è un metodo d'istanza, non statico.**
2. Si creano due oggetti `LibroDTO` usando il costruttore **senza** `id` :
 - **i libri non esistono ancora nel database, quindi l' `id` non è ancora noto.**
3. Si invoca `libroDao.insert()` per ognuno dei due libri.
 - **Il metodo restituisce un** `boolean` **che viene salvato in** `ok` **e** `ok2` **—**
`true` **se l'inserimento è andato a buon fine,** `false` **altrimenti.**
4. Se entrambi gli inserimenti sono riusciti, si stampa l' `id` di ciascun libro.

Nota

`libro1.getId()` e `libro2.getId()` restituiscono un valore valido — perché ricordi che nel metodo `insert()` del `LibroDAO`, dopo aver recuperato la chiave generata dal database tramite `getGeneratedKeys()`, abbiamo aggiornato il DTO con `l.setId(chiave)`. In questo modo il `LibroDTO` riflette lo stato reale del database.

5. Infine viene invocato `StampaLibri()`, definito come metodo separato:



Java

```

1 public static void StampaLibri() {
2     try {
3         List<LibroDTO> libri = LibroDAO.findAll();
4         System.out.println("=====LISTA LIBRI=====");
5         if (libri.isEmpty()) {
6             System.out.println("Lista vuota");
7         } else {
8             for (LibroDTO l : libri) {
9                 System.out.println("id: " + l.getId() + " titolo: "
10                    + l.getTitolo() +

```



```

10         " autore: " + l.getAutore() +
    " prezzo: " + l.getPrezzo());
11     }
12 }
13 } catch (SQLException e) {
14     e.printStackTrace();
15 }
16 }

```

`findAll()` è:

- un metodo statico del `LibroDAO`, quindi viene chiamato direttamente sulla classe senza istanziare nulla.
- Il try-catch è obbligatorio perché:
- `findAll()` dichiara throws `SQLException` nella sua firma

☞ `SQLException` è una checked exception e il compilatore ci obbliga a gestirla esplicitamente.

L'eccezione si propaga da `Database.getConnection()` fino a `findAll()` e infine fino a `StampaLibri()`, dove viene finalmente catturata e gestita.